

Name: _____

Class: _____

Mr. Rawson • GCSE Computer Science



Year 11 2026

Programming 8 — Sub Programs — **ANSWERS**

Why we need them

You have been using sub programs since your first lesson. `print()`, `len()`, `int()` and `input()` are all sub programs that someone else wrote. You call each one by its name, give it what it needs in the brackets, and it does its job.

A **sub program** is a block of code with a name. You write it once, then run it from anywhere in your program just by writing its name. Today you write your own.

VOCAB Sub program (OCR 2.2.3) — a named block of code that does one job. It runs when the program calls it by name. Procedures and functions are both sub programs

What it gives you	How
Write it once, use it many times	The code lives in one place; every call reuses it.
Easier to read	<code>show_menu()</code> says what happens in two words.
Easier to test	Each sub program can be run and checked on its own.
Easier to change	Fix or improve it in one place and every call gets the change.
Work in a team	Different programmers can write different sub programs.

These are the answers the exam wants when it asks for an advantage of sub programs. Breaking a big problem into sub programs is **decomposition**, from 2.1.

1 A procedure

The rule. Start with `def`, then the name, then empty brackets and a colon. Indent every line that belongs to the procedure. Then **call** it by writing its name and brackets, with no indent. Put the `def` above the call, so Python has read the procedure before it is asked to run it.

```
def show_menu():
    print("1 - New game")
    print("2 - Load game")
    print("3 - Quit")

show_menu()
```

Line	What it does
<code>def show_menu():</code>	Defines a procedure called <code>show_menu</code> . Nothing prints yet: Python only remembers the code.
The three print lines	Indented once, so they belong to the procedure.
<code>show_menu()</code>	The call . Python jumps into the procedure, runs its three lines, then carries on from the line after the call.

It prints the three menu lines. Write `show_menu()` twice and the menu prints twice, from code written once.

Now try • Write a procedure called `welcome` that asks the user for their name, then prints a welcome message using it. Call it **twice**.

Check • Enter Ana, then Ben. It prints Hello Ana, welcome to Computer Science. and then the same for Ben.

ANSWER

```
def welcome():
    name = input("What is your name? ")
    print("Hello " + name + ", welcome to Computer Science.")

welcome()
welcome()
```

2 Parameters: giving a sub program what it needs

`len("hello")` works on any word because you hand it the word in the brackets. Your own sub programs can take values in the same way.

The rule. Name the values the procedure needs inside the brackets of the `def` line. These names are its **parameters**. When you call it, put the actual values in the brackets, in the same order.

```
def siblings(brothers, sisters):
    total = brothers + sisters
    print("You have " + str(total) + " siblings.")

siblings(2, 1)
```

The call `siblings(2, 1)` puts 2 into `brothers` and 1 into `sisters`, first to first and second to second. It prints `You have 3 siblings.` Call `siblings(0, 4)` and the same code prints `You have 4 siblings.`

VOCAB Parameter (OCR 2.2.3) — a name in the brackets of a sub program that receives a value when the sub program is called

Now try • Write a procedure called `person` with two parameters, `name` and `age`. It prints the name and age in one sentence. Then, if the age is under 18, it prints `Too young to vote.`; otherwise it prints `Old enough to vote.`

Call it with `person("Ana", 15)` and `person("Ben", 40)`.

Check • Four lines: `Ana is 15. Too young to vote.` `Ben is 40. Old enough to vote.`

ANSWER

```
def person(name, age):
    print(name + " is " + str(age) + ".")
    if age < 18:
        print("Too young to vote.")
    else:
        print("Old enough to vote.")

person("Ana", 15)
person("Ben", 40)
```

3 A function hands a value back

`len("hello")` does not print anything. It works out 5 and hands it back to your program, which can store it or print it. A sub program that hands a value back is a **function**.

The rule. End the function with `return` and the value to hand back. Where you call it, catch the value: store it in a variable, or put the call straight inside `print()`.

```
def perimeter(length, width):
    perim = 2 * (length + width)
    return perim

p = perimeter(5, 7)
print(p)
print(perimeter(3, 4))
```

`perimeter(5, 7)` works out 24 and `return` hands it back to the line that called it, so `p` becomes 24. The last line calls it again and prints the result straight away: 14.

Procedure or function? One question decides it.

	Procedure	Function
Hands a value back?	No: it just does its job	Yes, with <code>return</code>
Called like this	<code>show_menu()</code> on its own line	<code>p = perimeter(5, 7)</code> or inside <code>print()</code>
Use it to	show a menu, print a report	work something out: a total, an area

VOCAB Function (OCR 2.2.3) — a sub program that returns a value to the line that called it. A procedure does its job and returns nothing

Now try • Write a function `circle_area(radius)` that returns the area of a circle. Use $\pi = 3.14$, so the area is `3.14 * radius ** 2`. Print the result for a radius of 10, then store the result for a radius of 2 in a variable called `small` and print `small`.

Check • 314.0, then 12.56.

ANSWER

```
def circle_area(radius):
    area = 3.14 * radius ** 2
    return area

print(circle_area(10))          # 314.0
small = circle_area(2)
print(small)                    # 12.56
```

4 Local and global variables

The rule. A variable made **inside** a sub program is **local**: it exists only while that sub program runs, and only the sub program can use it. A variable made in the **main program**, with no indent, is **global**: every part of the program can read it. To get a value **out** of a sub program, return it.

```
VAT = 0.2

def with_vat(price):
    extra = price * VAT
    return price + extra

print(with_vat(50))
```

Name	Scope	Why
VAT	global	Made in the main program. The function can read it. It is a constant : set once, never changed, so it is written in capitals.
price	local	A parameter belongs to its sub program.
extra	local	Made inside the function. When the function ends, it is gone.

It prints 60.0. Only the returned value leaves the function; extra stays inside it. Typing `print(extra)` in the main program gives a `NameError`, because the main program has no variable called extra.

Why local is the default. Each sub program keeps its own variables, so two sub programs can both use a name like `total` without changing each other's. That makes a program easier to test and to fix, and it is the advantage the exam asks about.

Now try • Predict first. The sub program has parameters with the **same names** as two global variables. Write down what the last two lines print, then run it.

```
var1 = 2
var2 = 3

def my_sub(var1, var2):
    var1 = var1 * 30
    var2 = var2 * 30
    return var2

var3 = my_sub(4, 5)
var4 = var1 + var2 + var3
print(var4)
print(var1)
```

Check • 155, then 2. Inside `my_sub`, `var1` and `var2` are local copies starting at 4 and 5, so the global 2 and 3 are untouched. `var3` is the returned 150, and $2 + 3 + 150 = 155$.

ANSWER

`my_sub(4, 5)`: local `var1` = $4 \times 30 = 120$, local `var2` = $5 \times 30 = 150$, returns 150. Back in the main program the globals are still 2 and 3, so `var4` = $2 + 3 + 150 = 155$, and `var1` is still 2.

Changing a global from inside a sub program. Write `global` and the name as the first line of the sub program. Then the sub program uses the main program's variable instead of making a local one.

```
score = 0

def add_point():
    global score
    score = score + 1

add_point()
add_point()
print(score)
```

It prints 2. Returning a value is usually the cleaner choice; `global` is there for a value the whole program shares, such as a game's score.

5 Pass an array in

The rule. An array can be a parameter, like any other value. Inside the function, loop over it exactly as in Programming 6, using the parameter's name.

```
def array_total(numbers):
    total = 0
    for i in range(len(numbers)):
        total = total + numbers[i]
    return total

scores = [5, 8, 2, 1, 9, 10, 7, 6]
print(array_total(scores))
```

The call hands scores to the function, where it is called numbers. The loop adds every box and return hands back 48. The same function totals **any** array: array_total([3, 4]) returns 7.

Now try • (a) Write a function average(numbers) that returns the average of an array. A function can call another function: use array_total(numbers) for the total and divide by len(numbers).

Check • print(average(scores)) prints 6.0.

(b) Write a function biggest(numbers) that returns the largest value. Start big at numbers[0], then loop through the rest, and whenever a box is larger than big, store it in big.

Check • biggest(scores) returns 10, and biggest([-4, -9, -2]) returns -2.

(c) Print both results in one sentence.

Check • Average 6.0, highest 10.

ANSWER

```
def average(numbers):
    return array_total(numbers) / len(numbers)

def biggest(numbers):
    big = numbers[0]
    for i in range(1, len(numbers)):
        if numbers[i] > big:
            big = numbers[i]
    return big

print("Average " + str(average(scores)) + ", highest " + str(biggest(scores))
      + ".")
```

Starting big at numbers[0] rather than 0 is what makes the all-negative array return -2. range(len(numbers)) is also correct for the loop; starting at 1 just skips comparing box 0 with itself.

Stretch A shop till

A shop till is decomposed into four sub programs: scan barcodes, look up prices, calculate the total, calculate change. Write the two that do the arithmetic.

1. `till_total(prices)` takes an array of prices **in pence** and returns the total.
2. `change(total, paid)` returns the change in pence.
3. The main program stores `[120, 85, 250]`, asks how much the customer paid in pence, and prints the total and the change using your two functions.

Check • Total 455. Pay 500 and the change is 45.

ANSWER

```
def till_total(prices):
    total = 0
    for i in range(len(prices)):
        total = total + prices[i]
    return total

def change(total, paid):
    return paid - total

basket = [120, 85, 250]
total = till_total(basket)
paid = int(input("Amount paid in pence: "))
print("Total: " + str(total))
print("Change: " + str(change(total, paid)))
```

Pence keep every value an Integer, so the arithmetic is exact. This is Paul Long's 7-mark question (p.96) cut down to two functions.

The exam writes it differently

OCR's exam language, ERL, uses a different word for each kind of sub program, and closes each one with its own end line instead of using the indent.

ERL (what you read)	Python (what you type)	Meaning
procedure show_menu() ... endprocedure	def show_menu():	A procedure
function perimeter(length, width) ... endfunction	def perimeter(length, width):	A function
return perim	return perim	Hand the value back
global score = 0	score = 0 in the main program	A global variable
p = perimeter(5, 7)	p = perimeter(5, 7)	Calling a function

The perimeter function in ERL.

```
function perimeter(length, width)
    perim = 2 * (length + width)
    return perim
endfunction

p = perimeter(5, 7)
print(p)
```

Python uses `def` for both kinds. In ERL, the word tells you which it is: if it says `function`, look for the `return`.